

İP ATLA — TECHNICAL DESIGN DOCUMENT (TDD) v1.0

Kaynak: GDD v2.0 (Temmuz 2026) · **Geliştirici:** Samet (tek kişi) · **Koşullar:** düşük bütçe, AI destekli geliştirme, ücretsiz araçlar, V1 odaklı

0. GDD EKSİK / BELİRSİZLİK / ÇELİŞKİ ANALİZİ

TDD'ye geçmeden önce tespit edilen ve bu dokümanda çözülen noktalar:

#	Tespit	Tip	Çözüldüğü bölüm
E1	Godot 4 web export'un SharedArrayBuffer (threads) sorunu GDD'de yok. Poki/CrazyGames/itch'te threads'li build çalışmayabilir.	Kritik risk	§1.3, §12.2
E2	Determinizm hedefi var ama simülasyon tick modeli tanımsız. ms pencereler frame-rate'e bağlanırsa replay/ghost bozulur.	Teknik belirsizlik	§4.1
E3	Poki SDK "analitik" diye geçiyor; oysa Poki'nin zorunlu gameplay event'leri + reklam API'si var ve JS interop gerektirir.	Eksik sistem	§12.3
E4	Web'de save kalıcılığı: <code>user://</code> IndexedDB'ye yazar; kullanıcı site verisi silerse veya oyun farklı domain'de barınırsa kayıt kaybolur. GDD'de yok.	Risk	§5.2
E5	"AI üretim + rötuş" karakter hattı araçsız/formatsız. AI-3D mesh çıktıları paylaşılan rig'e oturmaz.	Belirsizlik	§15.1
E6	Çember daralınca açısız yeniden dizilim kuralı tanımsız (oyuncu nasıl hep altta kalır, ip geçiş sırası nasıl korunur).	Eksik kural	§4.4
E7	Yüksek zıpla / eğil / çift tur etkileşim kuralları kesinleşmemiş (yüksek zıplarken yüksek süpürme gelirse ne olur?).	Eksik kural	§4.3
E8	"Müzik temposu ip hızıyla oranlanır": Godot'ta <code>pitch_scale</code> tempo+perdeyi birlikte değiştirir; GDD hangisini istediğini söylemiyor.	Belirsizlik	§8.3
E9	§7.1 "düşük çözünürlüklü gölge haritası" ↔ 2018 Android'de 60fps hedefi çelişir. Gölge haritası mobil web'de pahalı.	Çelişki	§11.2
E10	"Godot 4 3D mobil tarayıcıda 60fps" test edilmemiş varsayım. Gün 1'de doğrulanmalı.	Riskli karar	§14.4, §18
E11	İzleme modunda kamera davranışı (zoom mu, aynı sabit kamera mı) tanımsız.	Küçük eksik	§4.9
E12	Bot input'u ile insan input'unun aynı buffer/pencere kod yolundan geçip geçmediği belirsiz — adalet ve determinizm için aynı yol şart.	Teknik belirsizlik	§4.6

Tek kişi için gerçekçi olmayan yapı **bulunmadı** — GDD kapsam disiplini iyi. Tek şişme riski karakter üretim hattı (E5), §15'te sınırlandırıldı.

1. GENEL TEKNİK MİMARİ

1.1 Motor kararı: Godot 4.4 (GDScript)

Kriter	Godot 4	Unity	Phaser/Three.js	Defold
Web export	Birinci sınıf, tek tık	WebGL büyük (30-50MB+), mobil web zayıf	Doğal web ama 3D iskeleti elle kurarsın	Küçük build ama 3D desteği zayıf
2.5D low-poly	Hafif 3D pipeline, tam isabet	Aşırı güçlü = aşırı yük	glTF+animasyon yönetimi manuel	Uygun değil
Maliyet	%100 ücretsiz, MIT	Lisans/Runtime politika riski	Ücretsiz	Ücretsiz
AI destekli geliştirme	GDScript basit, LLM'ler iyi bilir	C# iyi bilinir ama boilerplate fazla	JS iyi bilinir	Lua, örnek havuzu dar
Android geçişi	Aynı proje, ikinci preset	Mümkün	Cordova/Capacitor sarmalama = ek iş	Mümkün

Karar: Godot 4.4, GDScript. Kritik teknik zorunluluk: **Godot 4'te C# web export YOK** — GDScript tek yol ve bu proje için yeterli (performans-kritik iş yok, fizik yok).

1.2 Renderer: Compatibility (WebGL2)

Forward+ ve Mobile renderer web'de desteklenmez. **Tüm platformlarda Compatibility** kullanılacak (web + Android aynı renderer = tek görsel doğrulama). Unlit/flat shading zaten hedef; Compatibility'nin kısıtları projeyi etkilemez.

1.3 Web export modu: threads KAPALI

Threads'li export SharedArrayBuffer ister → host sitede COOP/COEP header'ları gerekir. itch.io'da açılabilir ama **Poki ve CrazyGames'te garantisi yok**. Karar: **non-threaded export** (Godot 4.3+ destekliyor). Bedeli: ses gecikmesi biraz artar, yüklemede ana thread bloklanır — bu oyun için kabul edilebilir. Bu karar gün 1'de export preset'ine yazılır, sonradan değiştirmek test tekrarı demektir.

2. PROJE YAPISI

res://

```
└─ autoload/           # EventBus.gd, Config.gd, SaveGame.gd, Audio.gd,
                        # Analytics.gd, Ads.gd, Rng.gd
└─ scenes/
    └─ main.tscn       # Kök sahne (tek sahne mimarisi, §7.1)
    └─ arena/          # arena.tscn, rope_spinner.tscn, camera_rig.tscn
    └─ jumper/         # jumper.tscn (+ ragdoll.tscn)
        └─ ui/         # hud.tscn, main_menu.tscn, results.tscn,
                        # characters.tscn, settings_popup.tscn
└─ scripts/
    └─ core/           # game_state.gd, round_director.gd, tick_clock.gd
    └─ rope/           # rope.gd, rope_behavior.gd (Resource tabanı)
    └─ jumper/         # jumper.gd, input_source.gd, human_input.gd,
                        # bot_brain.gd, ghost_input.gd (v2a)
        └─ ui/
└─ data/
    └─ behaviors/      # *.tres (ip davranışları)
    └─ archetypes/     # *.tres (bot arketipleri)
        └─ characters/ # *.tres (koleksiyon karakterleri)
└─ config/
    └─ balance.json    # TÜM tuning değerleri (§5.1)
└─ assets/
    └─ models/ └─ textures/ └─ audio/{music,sfx}/ └─ fonts/
└─ addons/gut/        # test framework
```

Kurallar: dosya/klasör (snake_case); sınıflar (class_name PascalCase); sinyaller geçmiş zaman (jumper_eliminated); sahne başına bir script; GDScript'te namespace yok — class_name + klasör disiplini yeterli.

3. KOD MİMARİSİ

3.1 Autoload katmanı (7 singleton, daha fazlası yasak)

Autoload	Sorumluluk
EventBus	Global sinyaller (aşağıdaki tablo). Sadece sinyal tanımı, mantık yok.
Config	balance.json'ı yükler, tipli erişim sağlar (Config.perfect_window_ms).
SaveGame	user:// JSON okuma/yazma, migration (§5.2).
Audio	Bus yönetimi, SFX havuzu, müzik (§8).
Analytics	Event kuyruğu → aktif platform adaptörü (§13.4).
Ads	AdsService arayüzü: PokiAds / CrazyAds / NullAds (itch) (§12.3).

3.2 EventBus sinyali sözlüğü (v1'in tamamı)

gdscript

```
signal round_started(round_no: int, seed: int)
signal rope_crossed(jumper_id: int, result: int, delta_ms: float) # result: PER
signal jumper_stumbled(jumper_id: int)
signal jumper_pardoned(jumper_id: int)
signal jumper_eliminated(jumper_id: int, cause: int)
signal ring_shrunk(alive_count: int)
signal behavior_telegraphed(behavior_id: StringName)
signal behavior_started(behavior_id: StringName)
signal round_ended(placement: int, coins: int)
signal player_eliminated(alive_count: int) # izleme modu kararını tetikler
```

İletişim kuralı: **"call down, signal up."** Ebeveyn çocuğu doğrudan çağırır; çocuk yukarıya/uzaya EventBus sinyaliyle konuşur. Node'lar birbirini `get_node("../..")` ile aramaz.

3.3 Durum makineleri

GameState (oyun akışı): `MENU → COUNTDOWN → PLAYING → (SPECTATE) → RESULTS → COUNTDOWN...` Basit enum + `match` yeterli; framework/addon kullanılmaz.

RopeState (§4.2): davranış havuzunun kalbi, ayrı bölümde.

3.4 Jumper kompozisyonu (v2 sigortası)

gdscript

```
class_name Jumper extends Node3D
@export var input_source: InputSource # HumanInput | BotBrain | GhostInput
# durum: angle_pos, is_airborne, jump_started_tick, is_ducking,
# has_warning(▲), pardon_counter, is_alive
```

`InputSource.poll(tick) -> Array[InputEvent]` sözleşmesi: insan, bot ve (v2a) ghost aynı **Jumper kodundan** geçer. Bot avantajı/dezavantajı imkânsızlaşır (E12 çözümü), determinizm korunur, v2b'de `NetworkInput` eklemek tek sınıf demektir.

3.5 SOLID — pragmatik uygulama

Tek kişilik projede SOLID dogma değil filtre: (S) her script tek iş; (O) yeni ip davranışı = yeni `.tres` + gerekiyorsa küçük alt sınıf, rope.gd'ye dokunulmaz; (D) Jumper somut

input'a değil `InputSource` soyutlamasına bağlı. Interface segregation/Liskov için ekstra soyutlama **kurulmaz** — gereksiz karmaşıklık yasağı.

4. GAMEPLAY MİMARİSİ

4.1 Simülasyon çekirdeği: sabit tick (E2 çözümü)

- Tüm oyun mantığı `_physics_process`'te, **60Hz sabit tick** (`Engine.physics_ticks_per_second = 60`).
- Zaman birimleri config'te ms olarak yazılır, yüklemede tick'e çevrilir ($\pm 90\text{ms} \rightarrow \pm 5.4 \rightarrow 5$ tick; yuvarlama kuralı: `floor`, oyuncu lehine değil sabit).
- Girdiler `_input`'ta yakalanır, **tick numarasıyla damgalanıp** kuyruklanır; simülasyon yalnız tick kuyruğunu okur.
- Görsel yumuşaklık: ipin render açısı `_process`'te interpolasyonla çizilir; mantık açısı tick'te ilerler.
- Sonuç:** aynı seed + aynı input tick listesi = aynı tur, her cihazda. Replay, ghost (v2a) ve "bug raporu = seed" hedefleri bu blokla sağlanır.

4.2 İp sistemi — RopeState FSM

Durum verisi: `angle, angular_vel, height (LOW/HIGH), mode, telegraph_ticks_left, behavior_queue`.

Davranışlar veri odaklı: `RopeBehavior` (Resource):

```
id, min_round, weight, telegraph_ms, telegraph_anim, params(Dictionary)
```

Havuzdaki 7 davranış (GDD §4.1) bu şablonun parametre setleridir; **yalnız DoubleSweep ve FakeSlow** küçük özel mantık ister (alt sınıf). Genişleme havuzu (yanan ip vb.) = yeni `.tres` + shader/efekt $\rightarrow$ "1-2 saatlik iş" iddiası mimari olarak doğrulanır.

Davranış seçimi her N tick'te `RoundDirector` tarafından: ağırlıklı rastgele (`Rng.behavior` stream'i), kural filtreleri (aynı davranış max 2 ardışık; `min_round` kontrolü; zorluk eğrisi tablosu config'te).

4.3 Geçiş (crossing) ve zamanlama çözümü

Fizik motoru yok. Her tick:

```
gdscript
```

```

# rope.gd - tick()
var prev := angle
angle = wrapf(angle + angular_vel * TICK_DT, 0.0, TAU)
for j in alive_jumpers:
    if _swept_past(prev, angle, j.angle_pos, direction):
        _resolve_crossing(j)

func _resolve_crossing(j: Jumper) -> void:
    if height == HIGH:
        # yüksek süpürme
        if j.is_ducking: return # temiz geçiş (perfect sayılmaz, n
        if j.is_airborne: return _hit(j) # E7 kuralı: havadayken yüksek ip
        return _hit(j)
    # normal (yer hizası) süpürme:
    if not j.is_airborne: return _hit(j)
    var delta_ms := absf(current_tick - j.jump_input_tick) * TICK_MS
    if delta_ms <= Config.perfect_ms(round_no): _perfect(j, delta_ms)
    elif delta_ms <= Config.graze_ms(round_no): _graze(j)
    else: _hit(j)

```

Kesinleştirilen kurallar (E7):

- Zıplama havada kalma süresi: normal (`jump_air_ms`) (~420ms), yüksek zıplama (`high_jump_air_ms`) (~700ms) — config'te.
- **Çift tur:** iki süpürme arası < (`high_jump_air_ms`) olacak şekilde ayarlanır; tek yüksek zıplama ikisini örter. Sınıflandırma ilk geçişte yapılır; ikinci geçişte yalnız (`is_airborne`) kontrolü (havadaysa geçer, indiyse ıskalama).
- **Input buffer (100ms):** erken basış zaten (`graze`) penceresi içinde ölçülür; penceresinden de erken basış = erken zıplama, karakter ip gelmeden yere inerse ıskalama. Ek buffer mantığı gerekmez — pencere modeli bufferı kapsar; config'teki (`input_buffer_ms`) yalnız "zıplama animasyonu başlatma" toleransı olarak kalır.
- Eğilme süresi: basılı tutulduğu sürece + (`duck_release_ms`) çıkış toleransı.

4.4 Çember ve daralma (E6 çözümü)

- Jumper pozisyonu polar: (`ring_radius, angle_pos`); dünya koordinatı her tick türetilir.
- Yarıçap: $r = \text{lerp}(r_{\min}, r_{\max}, (\text{alive} - 2) / 14.0)$ — 16 kişide (`r_max`), 2 kişide (`r_min`).
- Eleme sonrası **yeniden dizilim:** kalan N jumper eşit açıyla ($\frac{\text{TAU}}{N}$) yeniden dağıtılır, ardından tüm dizilim oyuncu (270°)'de (ekranın altı) kalacak şekilde döndürülür. Geçiş 0.6 sn tween (yarıçap + açı birlikte), ip bu sırada dönmeye devam eder ama **crossing çözümü tween bitene kadar askıya alınır** (adalet).
- İp geçiş sırası doğal olarak açısal dizilimden gelir; ekstra sıra verisi tutulmaz.

4.5 Sendeleme sistemi

Jumper alanları: `(has_warning: bool, pardon_counter: int)`. Kurallar birebir GDD §3.2; rejim geçişleri (tur 20-29 daraltılmış, 30+ sudden death) config tablosundan okunur:

json

```
"stumble_regimes": [  
  {"from_round": 1, "graze_ms": [90,160], "pardon_rounds": 5},  
  {"from_round": 20, "graze_ms": [90,130], "pardon_rounds": 7},  
  {"from_round": 30, "graze_ms": null, "pardon_rounds": null}  
]
```

`(pardon_counter)` her temiz geçişte artar, sendelemede sıfırlanır; eşikle ulaşınca Δ silinir + `(jumper_pardoned)` sinyali (UI/ses tepkisi).

4.6 Bot beyni

`(BotBrain extends InputSource)`. Arketip = Resource:

```
reaction_mean_ms, reaction_std_base_ms, std_round_slope,  
telegraph_fail_prob, quirk (NONE/SHOWOFF/COPYCAT), exit_round_range
```

Karar modeli: ip botun konumuna `(lookahead_ms)` kala bot bir "niyet tick'i" örnekler: `(t_input = t_cross + N(reaction_mean,  $\sigma$ (round)))` — `(Rng.bots)` stream'inden.

- **Acemi:** yüksek σ + `(exit_round_range)` [3,8] içinde σ yapay şişirilir (gidiş garantiye alınır ama script'li ölüm yok, hâlâ zar atılır).
- **Panikçi:** telegraf sinyali geldiği turda `(telegraph_fail_prob=0.6)` ile $\sigma \times 3$.
- **Sağlam:** düşük σ , tur numarasıyla çok yavaş bozulur.
- **Şovcu:** perfect'te %30 takla animasyonu; takla turunda $\sigma \times 2$ ("taklaya takılır").
- **Kopyacı:** `(t_input = oyuncunun_input_tick + 200ms)`; oyuncu zıplamazsa kendi (kötü) örneğine düşer.

Bot çıktısı da `([tick, JUMP])` event'idir → §4.3'teki aynı çözümden geçer (E12).

4.7 Dinamik dram (RoundDirector içinde)

- `(player_early_exit_streak)` SaveGame'de tutulur (son 2 tur, "ilk 4 elemeye gitti" tanımı).
- Streak ≥ 2 ise: sonraki turda tur 1-3 arasında bir Acemi botun σ 'sı zorla şişirilir → ilk elenen bot olur.
- Final 1v1 botu: kalan botlar arasından arketipi Sağlam olana öncelik; yoksa en düşük σ 'lı bot terfi eder.
- Bu sistem `(Rng.bots)` stream'ini bozar mı? Hayır — müdahale de deterministik kuraldır, seed'e girdi olarak `(player_early_exit_streak)` eklenir (replay meta verisinde

saklanır).

4.8 Determinizm sözleşmesi (E2 devamı)

- `Rng` autoload'unda 3 bağımsız `RandomNumberGenerator`: `behavior`, `bots`, `cosmetic` (konfeti, seyirci animasyonu — gameplay'i etkilemez, senkron gerektirmez).
- Tur başlangıcı: `master_seed` → üç stream'e türetilmiş seed'ler (`hash(master_seed, "bots")`) vb.).
- Replay formatı: `{master_seed, streak_meta, inputs: [[tick, action], ...]}` → v2a ghost dosyası budur, ek iş sıfır.
- **Yasaklar:** gameplay kodunda `randf()` (global RNG), `Time.get_ticks_msec()` ile mantık, `_process`'te durum değiştirme, `Dictionary` iterasyon sırasına güvenme.
- Float determinizmi: aynı platformda garanti; platformlar arası (web↔Android ghost) küçük sapma riski var — v2a'da ghost'lar platform içi eşleştirilir, sorun ertelenir.

4.9 Eleme ve izleme

- Jumper normalde `Node3D` (fizik yok). Elemede: görsel mesh `ragdoll.tscn`'e (`RigidBody3D` + basit joint'ler ya da tek gövde + açılabilir impuls — **v1'de tek gövde impuls yeter**, gerçek ragdoll polish fazında denenir) devredilir, kameraya/derinliğe rastgele (`Rng.cosmetic`) fırlatılır, 2 sn sonra freeze + tribün noktasına ışınlanıp seyirci animasyonuna geçer.
- Slow-motion: `Engine.time_scale = 0.3`, 0.4 sn (gerçek zaman timer'la geri alınır). Tick determinizmini bozmaz (tick sayısı değişmez, sadece gerçek-zaman esner) — ama **girdi damgalama tick bazlı olduğundan** oyuncu bu pencerede avantaj kazanmaz.
- İzleme modu (E11): aynı sabit kamera, yalnız FOV %10 daralır + hedef final ikilisinin orta noktasına hafif pan. Yeni kamera sistemi yazılmaz. `[ATLA]` butonu Results'a geçer.

5. VERİ YÖNETİMİ

5.1 Config — `res://config/balance.json`

Tek dosya, oyun açılışında `Config` yükler. Alanlar (tamamı):

json

```
{
  "version": 1,
  "timing": { "perfect_ms": 90, "graze_ms": 160, "input_buffer_ms": 100,
    "jump_air_ms": 420, "high_jump_air_ms": 700,
    "hold_threshold_ms": 300, "duck_release_ms": 120 },
  "rope": { "start_rpm": 25, "speed_step_ratio": 1.08, "max_rpm": 60 },
  "stumble_regimes": [ ... §4.5 ... ],
  "difficulty_rounds": [ {"from": 1, "behaviors": ["normal"]},
    {"from": 6, "behaviors": ["speed_step", "sudden_stop"]} ],
  "ring": { "r_max": 9.0, "r_min": 2.2, "shrink_tween_s": 0.6 },
  "economy": { "win": 50, "top3": 25, "top8": 10, "other": 5,
    "perfect_bonus": 1, "daily_first_win_mult": 3,
    "ad_mult": 2, "gacha_cost": 100 },
  "bots": { "archetype_counts": {"acemi": 4, "panikci": 3, "saglam": 3,
    "sovcu": 2, "kopyaci": 2, "lookahead_ms": 600 } },
  "drama": { "early_exit_threshold": 4, "streak_len": 2 }
}
```

Neden JSON, neden Resource değil: elle düzenleme + diff kolaylığı, playtest sırasında editor açmadan tuning, başka bir AI'a "şu değeri değiştir" demenin en ucuz formatı. Karakter/davranış/arketip tanımları ise **Resource (.tres)** — çünkü mesh/animasyon referansı taşırlar.

5.2 Save — `user://save.json`

json

```
{ "version": 1, "coins": 0, "characters_owned": ["default"],
  "equipped": "default", "best_round": 0, "total_wins": 0,
  "last_daily_win_date": "", "early_exit_streak": 0,
  "settings": { "sound": true, "vibration": true, "lang": "en",
    "orientation": "portrait", "reduced_fx": false } }
```

- Yazım: her tur sonu + ayar değişiminde; önce temp dosyaya yaz, sonra rename (yarım yazım koruması).
- `version` + migration fonksiyon zinciri (v1→v2...) ilk günden kurulur.
- **E4 riski:** web'de `user://` = IndexedDB; origin değişirse (itch → Poki) kayıt taşınmaz. Kabul edilen kayıp: v1 ekonomisi hafif, kayıt kaybı ölümcül değil. Ayarlar ekranına "web sürümünde ilerleme bu tarayıcıya bağlıdır" notu. Cloud save v1'de YOK.
- **SQLite yok:** ilişkisel veri yok, tek profil, ~1KB veri. Gereksiz bağımlılık eklenmez.

6. INPUT SİSTEMİ

Aksiyon (InputMap)	PC	Mobil
<code>jump</code>	SPACE / Sol tık	Tek dokunuş
<code>high_jump</code>	SPACE basılı ≥300ms	Basılı tutma ≥300ms
<code>duck</code>	S / ↓	Aşağı kaydırma (playtest alternatifi: ekran alt yarısı basılı tut)

- Yakalama `_input()`'ta; her event `{tick, action, pressed}` olarak `InputQueue`'ya. `HumanInput.poll(tick)` kuyruğu boşaltır.
- Hold tespiti: `jump` basılı kalma süresi tick'le ölçülür; 300ms eşiği aşılsa havadayken zıplama yüksekliği yükseltir (basışta değil — geç karar oyuncu lehine).
- Duck kaydırması: `InputEventScreenDrag`, dikey delta > 60px & aşağı yön.
- Gamepad v1'de YOK (web+mobil hedefi; SPACE zaten A tuşuna maplenebilir, v1.1 işi 10 dk).
- Mobil gecikme ölçümü (GDD açık soru 1):** `Analytics`'e her crossing'de `delta_ms` yazılır; platform bazında dağılım karşılaştırılır → pencereler platform başına offset alabilir (`touch_latency_offset_ms` config alanı şimdiden konur, default 0).

7. UI MİMARİSİ

7.1 Tek sahne mimarisi

`main.tscn` hiç değişmez; içinde Arena3D + `CanvasLayer` katmanları:

```
Main
├─ Arena3D (SubViewport YOK - direkt)
├─ UILayer (CanvasLayer)
│   ├── MainMenu   ├── HUD       ├── Results
│   ├── Characters  ├── SettingsPopup
├─ FadeLayer (CanvasLayer, geçiş karartması)
```

Restart = sahne reload DEĞİL, `RoundDirector.reset()` (jumper durumları, ip, sayaçlar). <2 sn restart garantisi buradan gelir — sahne yükleme maliyeti sıfır. Ekranlar `visible` toggle + tek `UIRouter` scripti.

7.2 Oryantasyon (dikey/yatay)

- Project settings: `stretch_mode = canvas_items`, `aspect = expand`.
- `get_viewport().size_changed` sinyali → `aspect < 1` ise portrait preset, değilse landscape.
- CameraRig iki preset (pozisyon/FOV/tilt) arasında tween; UI container'ları anchor tabanlı — portrait'te alt/üst şerit `VBoxContainer`, landscape'te köşe `MarginContainer`'ları. İki ayrı layout sahnesi YAPILMAZ; aynı node'lar anchor preset değiştirir.

7.3 HUD elemanları

Kalan oyuncu sayacı (eleme sinyalinde pulse tween), perfect combo etiketi, $\triangle$ durumu = ekran kenarı kırmızı çerçeve (tam ekran `Panel` + shader yerine 4 ince `ColorRect`), ucuz, "DOKUN/TAP" ilk zıplamaya kadar. Popup sistemi: tek `SettingsPopup`, modal karartma `ColorRect`.

8. AUDIO SİSTEMİ

8.1 Bus yapısı

`Master → Music, SFX`. Ayarlardaki "ses" toggle'ı Master mute (v1'de ayrı slider yok — sadelik).

8.2 SFX

`Audio.play(sfx_name, pitch_var)` — 8 adet `AudioStreamPlayer` havuzu (round-robin). "Vuş" metronomu `rope_crossed` değil **süpürme açısı oyuncu bölgesinden geçerken** tetiklenir (crossing sonucu beklenmez, ritim bilgisi önden gelir). Telegraf jingle'ları `behavior_telegraphed` sinyalinden.

8.3 Müzik (E8 kararı)

`pitch_scale = rope_rpm / start_rpm` ile perde+tempo birlikte yükselir. Chiptune/percussive loop'ta bu **istenen** etkidir (gerilim); time-stretch kütüphanesi eklenmez. Loop, perde kayması çirkinleşmesin diye melodik değil ritmik seçilir (asset kriteri §15.3'e not).

9. AI SİSTEMİ KARARI

Yaklaşım	Uygunluk
FSM	Botun "durumları" yok — tek karar: ne zaman zıplayacağı

Yaklaşım	Uygunluk
Behavior Tree	Ağaç kuracak dallanma yok; addon bağımlılığı + öğrenme maliyeti
GOAP / Utility	Hedef/aksiyon uzayı tek boyutlu; topa tüfek
Stokastik reaksiyon modeli (§4.6)	SEÇİLDİ — arketip = 5-6 sayılık dağılım parametresi

Gerekçe: bu oyunun botu bir *zamanlama hatası üreticidir*, bir *ajan* değil. Tüm "kişilik" hata dağılımının şekli + birkaç quirk'tür. ~80 satır kod, tamamı deterministik, tamamı config'ten ayarlanır. Zorluk eğrisiyle senkron: $\sigma(\text{round})$ fonksiyonu difficulty tablosuyla aynı kaynaktan beslenir.

10. MULTIPLAYER

v1: YOK. Kod tarafında yalnız üç sigorta yaşar (maliyeti ~sıfır):

1. `InputSource` soyutlaması (§3.4) — `NetworkInput` yuvası hazır.
2. Deterministik tick + replay formatı (§4.8) — ghost/async altyapısı hazır.
3. İp durumu 5 alanlık serileştirilebilir struct — senkronize edilecek tek şey bu + zıplama event'leri.

v2a (async): replay dosyası + Supabase ücretsiz katman (skor tablosu, ghost depolama).

Godot HTTPRequest yeterli, SDK gerekmez. **v2b (gerçek zamanlı):** WebSocket + otoriter sunucu (Node.js/Go, tek oda tipi). Sunucu simülasyonu aynı GDScript mantığının portu olacağı için **gameplay kodunda Godot API'sine bağımlılığı azalt** (mantık saf fonksiyonlarda) — v1'de uyulacak tek ek disiplin. Detay tasarımı D1 tetikleyicisi sağlanmadan YAZILMAZ (GDD kuralına sadakat).

11. PERFORMANS

11.1 Bütçeler (2018 Android, Chrome, 60fps)

- Üçgen: sahne toplam <50k (karakter ~800 tris $\times$ 17 + arena ~20k).
- Draw call: <60. Yöntem: tüm karakterler **tek atlas + palet dokusu** (materyal 1 adet), arena obje materyalleri birleştirilir.
- Işık: 1 directional, **gölge haritası KAPALI** (E9 çözümü). Karakter gölgesi = altına yapışık blob quad (unlit, alpha). GDD'nin "düşük çözünürlüklü gölge haritası" ifadesi iptal — mobil WebGL'de en pahalı kalemdi.
- Post-process YOK. Glow/parlaması gereken şeyler (ip izi, perfect halkası) emissive doku + additive quad.

11.2 Pooling ve bellek

- Havuzlanan: konfeti/toz partikülleri (CPUParticles3D, 4 adet), SFX player'ları, floating text.
- Havuzlanmayan: 17 jumper sabittir, tur boyunca yaşar — instantiate/free döngüsü yok zaten.
- Ragdoll: tek sahne, eleme anında aktive, kullanım sonrası uykuya (`freeze = true`).
- Web build boyutu hedefi: <30MB sıkıştırılmış (Poki limiti ve mobil web yükleme toleransı). Doku: WebP/sıkıştırılmış, ses: OGG mono SFX.

11.3 Ölçüm

Debug overlay (§13.3) + Godot profiler; her fazın sonunda hedef cihazda 5 dk ölçüm ritüeli.

12. BUILD SİSTEMİ

12.1 v1 hedefleri

Platform	v1?	Not
Web (itch.io)	✓	İlk test kanalı, NullAds
Web (Poki)	✓ başvuru	PokiAds + zorunlu SDK event'leri
Web (CrazyGames)	✓ paralel başvuru	CrazySDK adaptörü
Android (Play)	v1.1	Aynı proje, ikinci export preset; 25\$ kayıt v1 verisi olumluysa
iOS	✗	Mac + 99\$/yıl + review — v2+

12.2 Web export preset

Threads OFF (§1.3) · Extensions OFF · VRAM texture compression: **for mobile ON** · Progressive loading splash · Boot süresi hedefi <8 sn (Poki kriteri).

12.3 SDK mimarisi (E3 çözümü)

gdscript

```
class_name AdService # arayüz
func gameplay_start() ; func gameplay_stop()
func rewarded(cb: Callable) # cb(success: bool)
func commercial_break(cb) # v1'de çağrılmaz (zorunlu reklam yok)
```

(PokiAds) / (CrazyAds) implementasyonları (JavaScriptBridge.eval/get_interface) ile portal SDK'sını sarar; (NullAds) (itch/Android-ilk) rewarded'ı anında (success=false) döndürür → UI ×2 butonunu gizler. Platform tespiti: export feature tag + URL kontrolü. **Poki'nin gameplayStart/Stop event'leri zorunludur** — (round_started/round_ended) sinyallerine bağlanır, unutulursa başvuru reddedilir.

13. HATA YÖNETİMİ

1. **Logger** (Config içinde basit fonksiyon, ayrı autoload şişkinliği yok): seviye (DEBUG/INFO/WARN/ERROR), her satıra ([seed:{s}][tick:{t}]) damgası → "bug raporu = seed" hedefi loglarla kapanır. Ring buffer (son 500 satır) bellekte; ERROR'da Analytics'e (client_error) event'i.
2. **Assert'ler**: geliştirmede (assert()) bol kullanılır (release build'de otomatik silinir): açılma aralıkları, alive sayısı, config alan varlığı.
3. **Debug overlay** (yalnız (OS.is_debug_build())): FPS, tick, seed, rope state, davranış kuyruğu, her botun o'sı. Tuş: F3. + hile tuşları: F4 tur atla, F5 seed gir.
4. **Analytics şeması**: (round_start{round, seed}), (elimination{round, cause, behavior, delta_ms}), (restart{}), (session_end{rounds}), (rewarded{shown, completed}), (client_error{msg}). Adaptörler: Poki SDK event'i / kendi endpoint (v1'de basit: Supabase ücretsiz tabloya HTTPRequest POST; kota aşımında sessiz düşer).

14. TEST STRATEJİSİ

Katman	Araç	Kapsam
Birim	GUT addon	Zamanlama sınıflandırması (sınır değerleri: 89/90/91ms...), crossing açılma matematiği (wrap, ters yön), af sayacı, ekonomi hesabı, save migration
Determinizm	GUT	Aynı seed+input → aynı eleme sırası ve tick'leri; 3 farklı FPS ayarında (30/60/144) sonuç değişmiyor
Balans	Headless sim	(godot --headless --script sim.gd): 16 bot, 1000 tur → CSV (tur uzunluğu, eleme eğrisi, arketip sağkalımı) → GDD §4.2 beklenen

Katman	Araç	Kapsam
		kadro tablosuyla karşılaştırma. Fizik olmadığı için ~saniyeler sürer; her balans değişikliğinde koşulur = regresyon testi
Manuel	Checklist	Oryantasyon geçişi, restart <2sn, izleme modu, reklam akışı (mock), save bozulması senaryosu
Cihaz	3'lü matris	Düşük Android+Chrome, iPhone+Safari (web), PC. Gün 1'de boş sahne + 17 kapsül FPS testi (E10) — geçemezse mimari revizyon oyun yazılmadan yapılır
Playtest	GDD Faz 3 protokolü	5 kişi; "haksızlık" kelimesi avı; analitik <code>delta_ms</code> dağılımı pencere tuning'ine girdi

15. ASSET PIPELINE

15.1 Karakterler (E5 çözümü)

- **Tek master rig:** Blender'da 1 kez — kapsül gövde, 8-10 kemik, 5 ortak animasyon (zıpla, sendele, savrul, sevin, üzül) tek `AnimationLibrary`'de, tüm karakterler paylaşır.
- Varyasyon = mesh swap + palet dokusu + aksesuar node'u (şapka/kafa objesi). Yeni karakter maliyeti: ~1-2 saat.
- AI rolü: **konsept görseli** (ücretsiz: Bing Image Creator / Leonardo free) → elle low-poly modelleme (**Blockbench** önerilir: ücretsiz, low-poly'e optimize, glTF export, öğrenmesi 1 akşam). AI-3D mesh servisleri (Meshy/Tripo) ana hat DEĞİL: çıktı topolojisi rig'e oturmaz, retopo süresi elle modellemeden uzun. Sadece ilham/istisna.
- Nadir karakter özel eleme animasyonu: rig animasyonu değil **efekt** (robot = parça sprite patlaması, dondurma = shader erime) — rig'e dokunmadığı için ucuz.
- Format: glTF (.glb).

15.2 Arena

Kenney asset paketleri (CC0): zemin, kaldırım, top, sıra. Renk paleti tek doku üzerinden — atlas disiplinine dahil.

15.3 Ses

Kenney Audio + freesound (yalnız CC0 filtresi; her dosyanın kaynağı `assets/audio/LICENSES.md`'ye yazılır). Müzik: ritmik/percussive loop (§8.3 kriteri) — OpenGameArt CC0 veya AI müzik (Suno ücretsiz katman, ticari kullanım şartını yayın öncesi doğrula; belirsizse CC0'a dön).

15.4 UI/Marketing

Font: Google Fonts (rounded, çok dilli). İkon + 3 ekran görüntüsü + 15 sn video: OBS kayıt +

16. VERSİYON KONTROLÜ

- Git + GitHub private. `.gitignore`: `.godot/`, `export/`, `*.tmp`.
 - Dallar: `main` (her commit'i export edilebilir) + `dev` (günlük iş). Solo için feature branch yalnız riskli deneylerde (ör. ragdoll denemesi) — merge etmezsen sil, `dev` kirlenmez.
 - Commit örnekleri: `feat:`, `fix:`, `balance:` (yalnız balance.json), `asset:`, `chore:`. `balance:` ayrı çünkü headless sim regresyonu bu commit'lerde koşulur.
 - Etiket: `v0.1.0` (Faz sonları), `v1.0.0` (yayın). Binary'ler küçük (<100MB toplam) — LFS gerekmez.
-

17. V1 TEKNİK KAPSAMI

Kesin yapılacak: Arena modu, 7 ip davranışı, sendeleme + af + rejimler, 5 bot arketipi + dinamik dram, daralan çember, deterministik tick + replay kaydı, jeton + gacha + 12 karakter, tek arena, TR/EN, web export (itch+Poki+CrazyGames), rewarded reklam, analitik, save.

Kesin yapılmayacak: gerçek multiplayer, IAP, cloud save, gamepad, görev/sezon/enerji sistemleri, gerçek ragdoll (tek gövde impuls yeter), gölge haritası, tutorial ekranı, iOS.

V2'ye ertelenen: ghost/async (altyapı hazır), Mod 2-3, günlük meydan okuma (deterministik yapı sayesinde ucuz — ilk güncelleme adayı), Android (v1.1), genişleme davranış havuzu.

Teknik riskler:

Risk	Olasılık	Erken sinyal	Önlem
Godot web mobilde 60fps tutmaz	Orta	Gün 1 FPS testi	Karakter tris düşür, 12 kişi moduna in, partikül kıs
Poki SDK entegrasyon reddi	Düşük-orta	Başvuru geri bildirimi	itch+CrazyGames paralel; SDK event'leri gün 1'den mimaride
Karakter üretimi süre yer	Orta	İlk karakter >3 saat sürerse	Launch 12→8 karaktere iner, haftalık ekleme telafi eder
IndexedDB kayıt kaybı şikâyeti	Düşük	—	Ayarlarda uyarı metni; ekonomi hafif tutuldu
pitch_scale müziği çirkinleştirir	Düşük	Faz 4 kulak testi	Yedek plan: 3 yoğunluk katmanı crossfade

18. GELİŞTİRME SIRASI (teknik aşamalar)

- İskelet: proje + autoload'lar + config yükleme + Git + web export preset
↳ AYNI GÜN: hedef cihazda boş sahne + 17 kapsül FPS testi (E10 kapısı)
- Tick çekirdeği: tick_clock + Rng stream'leri + input kuyruğu
- İp: RopeState (yalnız Normal) + crossing matematiği + süpürme gölgesi
- Jumper: zıplama + zamanlama sınıflandırma + input buffer
↳ GDD Faz 1 GEÇİT TESTİ (his) burada
- Sendeleme sistemi + eleme (tek gövde impuls) + daralan çember + yeniden dizilim
- Botlar: InputSource + 3 arketip + zorluk eğrisi bağlantısı
- Kalan ip davranışları (telegraf sistemiyle birlikte) + kalan 2 arketip + dinamik dram
- UI: tek sahne router + HUD + sonuç + restart reset + oryantasyon
- Save + ekonomi + gacha + karakter Resource sistemi (asset'ler paralel üretilir)
- Ses + müzik pitch bağlama + eleme sahnesi cilası + slow-motion
- Determinizm birim testleri + headless balans simülasyonu + tuning
- SDK katmanı: Ads arayüzü + Poki/Crazy adaptörleri + analitik
- Yerelleştirme (CSV → Godot translation) + ayarlar + "rakipler hakkında" metni
- Polish: partiküller, konfeti, kamera zoom, ekran görüntüleri
- Export + itch yayın + Poki/CrazyGames başvuru

Sıra bağımlılık grafiğidir: 2 olmadan 3'te determinizm kurulamaz; 6, 4'teki aynı kod yolunu kullanır; 12 en sona bırakılmaz çünkü Poki event'leri 8'deki akışa gömülür (sadece adaptör

sona kalır).

19. KAPANIŞ DEĞERLENDİRMESİ

1. Bu TDD ile V1 doğrudan geliştirilebilir mi? Evet. Mimari kararların tamamı verildi, veri şemaları yazıldı, sistemlerin çalışma mantığı ve bağımlılık sırası tanımlı. Başka bir AI/geliştirici §18 sırasını takip ederek üretime başlayabilir.

2-3. Kalan teknik kararlar var mı? Neler? Kritik (bloklamayan ama üretim sırasında kapanacak) 5 karar kaldı:

- Gün 1 FPS kapısı (E10):** hedef cihazda test sonucu mimariyi doğrular — tek gerçek bilinmeyen bu.
- Duck girdisi** (swipe vs alt-yarı basılı): playtest kararı, iki implementasyon da ~30 dk; ikisi de yazılıp toggle'lanabilir.
- Müzik tempo yöntemi** (pitch_scale vs katman crossfade): Faz 4'te kulakla, yedek plan hazır.
- İsim + domain/mağaza taraması:** teknik değil ama Poki başvurusunu bloklar.
- Suno/AI müzik lisans doğrulaması:** yayın öncesi; belirsizse CC0.

4. Üretime başlamadan önce mutlaka cevaplanması gerekenler:

- Hedef "en düşük cihaz" tam olarak hangisi? (Elindeki gerçek bir telefon seç, o referans olsun.)
- Poki başvuru kriterlerini (boot süresi, SDK event'leri, boyut) bugünkü halleriyle bir kez oku — §12 varsayımları güncel mi doğrula.
- İlk karakterin Blockbench'te üretim süresi ölç → 12 karakter takvimi gerçekçi mi?

Bunların dışında **kritik eksik kalmamıştır**. TDD, tek geliştirici + AI destekli üretim şartlarında V1'i taşıyacak yeterliliktedir.